# Notes on Chapter 18: Missing Values

Norman Lim

2025-08-03

### Prerequisites

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.4     v readr     2.1.5
## v forcats   1.0.0     v stringr   1.5.1
## v ggplot2   3.5.2     v tibble    3.3.0
## v lubridate 1.9.4     v tidyr     1.3.1
## v purrr     1.1.0
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

### Last observation carried forward

```
treatment <- tribble(
  ~person,           ~treatment, ~response,
  "Derrick Whitmore", 1,          7,
  NA,                 2,          10,
  NA,                 3,          NA,
  "Katherine Burke",  1,          4
)
```

- Using `fill()` to fill missing values by carrying the last observation forward aka **locf**:

```
treatment |>
  fill(everything())
```

```
## # A tibble: 4 x 3
##   person           treatment response
##   <chr>                <dbl>    <dbl>
## 1 Derrick Whitmore         1        7
## 2 Derrick Whitmore         2       10
## 3 Derrick Whitmore         3       10
## 4 Katherine Burke          1        4
```

### Filling missing values using fixed values:

- Using `coalesce()`:

```
x <- c(1, 4, 5, 7, NA)
```

```r
coalesce(x, 0)
```

```
## [1] 1 4 5 7 0
```

Sometimes, in the data, other values are used to represent missing values, like `99` or `-999`. We can replace them with `NA` using the `na_if()` function:

```r
x <- c(1, 4, 5, 7, -99)
na_if(x, -99)
```

```
## [1]  1  4  5  7 NA
```

### NaN

`NaN` generally behaves like `NA`:

```r
x <- c(NA, NaN)
x * 10
```

```
## [1]  NA NaN
```

```r
x == 1
```

```
## [1] NA NA
```

```r
is.na(x)
```

```
## [1] TRUE TRUE
```

- In case you need to distinguis `NA`, from `NaN`, use `is.nan()`:

```r
is.nan(x)
```

```
## [1] FALSE  TRUE
```

`NaN`s are usually produced when you perform a mathematical operation that has an indeterminate result:

```r
0 / 0
```

```
## [1] NaN
```

```r
0 * Inf
```

```
## [1] NaN
```

```r
Inf - Inf
```

```
## [1] NaN
```

```r
sqrt(-1)
```

```
## Warning in sqrt(-1): NaNs produced
```

```
## [1] NaN
```

## Implicit missing values

- Example: an entire row of data is missing.

```r
stocks <- tibble(
  year  = c(2020, 2020, 2020, 2020, 2021, 2021, 2021),
  qtr   = c(   1,    2,    3,    4,    2,    3,    4),
  price = c(1.88, 0.59, 0.35,   NA, 0.92, 0.17, 2.66)
)
```

```
stocks
```

```
## # A tibble: 7 x 3
##    year   qtr price
##   <dbl> <dbl> <dbl>
## 1  2020     1  1.88
## 2  2020     2  0.59
## 3  2020     3  0.35
## 4  2020     4 NA
## 5  2021     2  0.92
## 6  2021     3  0.17
## 7  2021     4  2.66
```

- In the dataset above, price in the fourth quarter of 2020 is explicitly missing because its value is `NA`
- The `price` for the first quarter of 2021 is implicitly missing because it does not appear in the dataset.

In other words:

> An explicit missing value is the presence of an absence. An implicit missing value is the absence of a presence.

### Pivoting

- Pivoting can make implicit missings explicit and vice versa:

```
stocks |>
  pivot_wider(
    names_from = qtr,
    values_from = price
  )
```

```
## # A tibble: 2 x 5
##    year   `1`   `2`   `3`   `4`
##   <dbl> <dbl> <dbl> <dbl> <dbl>
## 1  2020  1.88  0.59  0.35 NA
## 2  2021 NA     0.92  0.17  2.66
```

### Complete

`tidyr::complete()` allows you to generate explicit missing values by providing a set of variables that define the combination of rows that should exist. Example:

```
stocks |>
  complete(year, qtr)
```

```
## # A tibble: 8 x 3
##    year   qtr price
##   <dbl> <dbl> <dbl>
## 1  2020     1  1.88
## 2  2020     2  0.59
## 3  2020     3  0.35
## 4  2020     4 NA
## 5  2021     1 NA
## 6  2021     2  0.92
## 7  2021     3  0.17
## 8  2021     4  2.66
```

If the individual variables are not complete aka there is not enough example for `complete()` to draw from, you can specify the missing part:

```
stocks |>
  complete(year = 2019:2021, qtr)
```

```
## # A tibble: 12 x 3
##     year   qtr price
##    <dbl> <dbl> <dbl>
##  1  2019     1 NA
##  2  2019     2 NA
##  3  2019     3 NA
##  4  2019     4 NA
##  5  2020     1  1.88
##  6  2020     2  0.59
##  7  2020     3  0.35
##  8  2020     4 NA
##  9  2021     1 NA
## 10  2021     2  0.92
## 11  2021     3  0.17
## 12  2021     4  2.66
```

### Joins

The function `dplyr::anit_join(x, y)` is a useful tool in revealing rows in `x` that don't have match in `y`. Example:

```
library(nycflights13)

flights |>
  distinct(faa = dest) |>
  anti_join(airports)
```

```
## Joining with `by = join_by(faa)`
```

```
## # A tibble: 4 x 1
##   faa
##   <chr>
## 1 BQN
## 2 SJU
## 3 STT
## 4 PSE
```

```
flights |>
  distinct(tailnum) |>
  anti_join(planes)
```

```
## Joining with `by = join_by(tailnum)`
```

```
## # A tibble: 722 x 1
##    tailnum
##    <chr>
##  1 N3ALAA
##  2 N3DUAA
##  3 N542MQ
##  4 N730MQ
##  5 N9EAMQ
```

```
##  6 N532UA
##  7 N3EMAA
##  8 N518MQ
##  9 N3BAAA
## 10 N3CYAA
## # i 712 more rows
```

**Exercises**

1. Can you find any relationship between the carrier and the rows that appear to be missing from `planes`?

## Factors and empty groups

This can happen if the factors do not contain all the possible entries specified in the labels. Example:

```
health <- tibble(
  name = c("Ikaia", "Oletta", "Leriah", "Dahsay", "Tresaun"),
  smoker = factor(c("no", "no", "no", "no", "no"), levels = c("yes", "no")),
  age = c(34, 88, 75, 47, 56)
)

health |> count(smoker)
```

```
## # A tibble: 1 x 2
##   smoker     n
##   <fct>  <int>
## 1 no         5
```

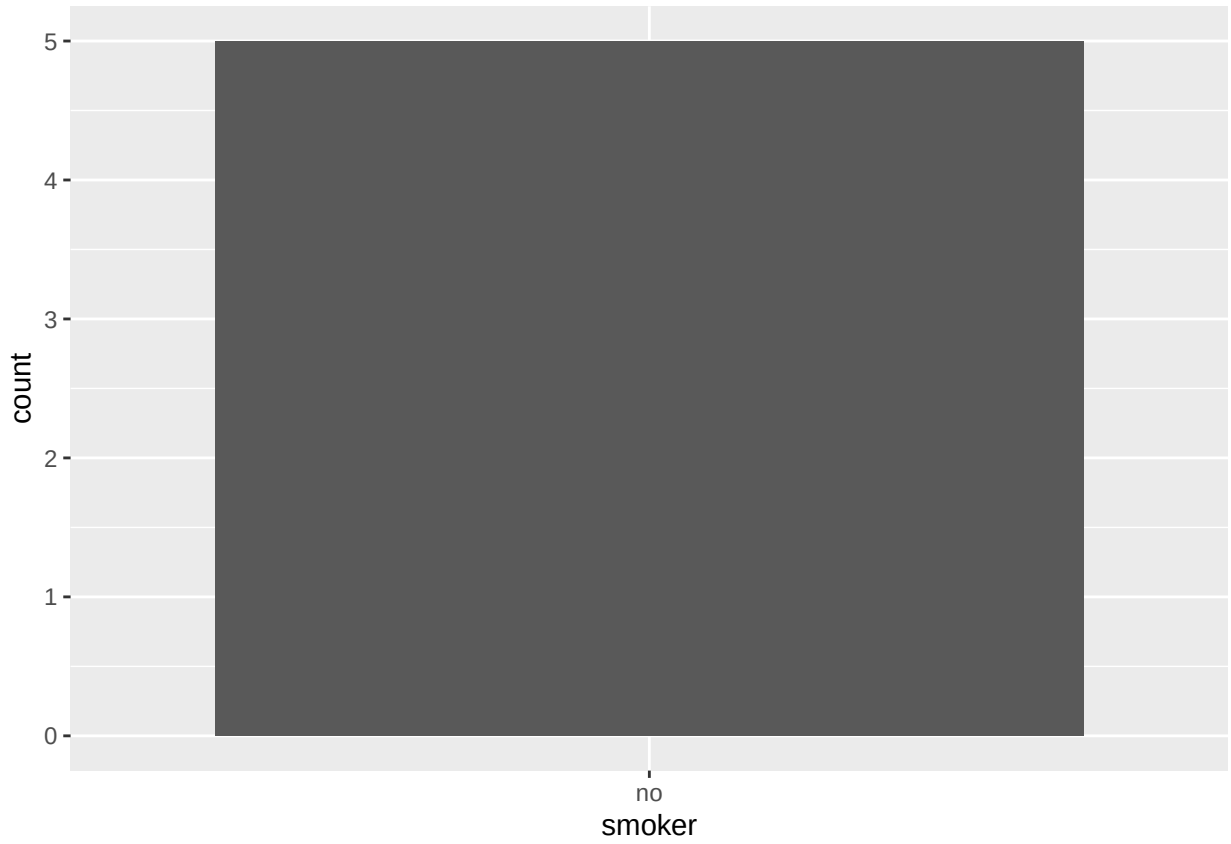
We can force `count()` to keep all the groups using `.drop = FALSE`:

```
health |> count(smoker, .drop = FALSE)
```

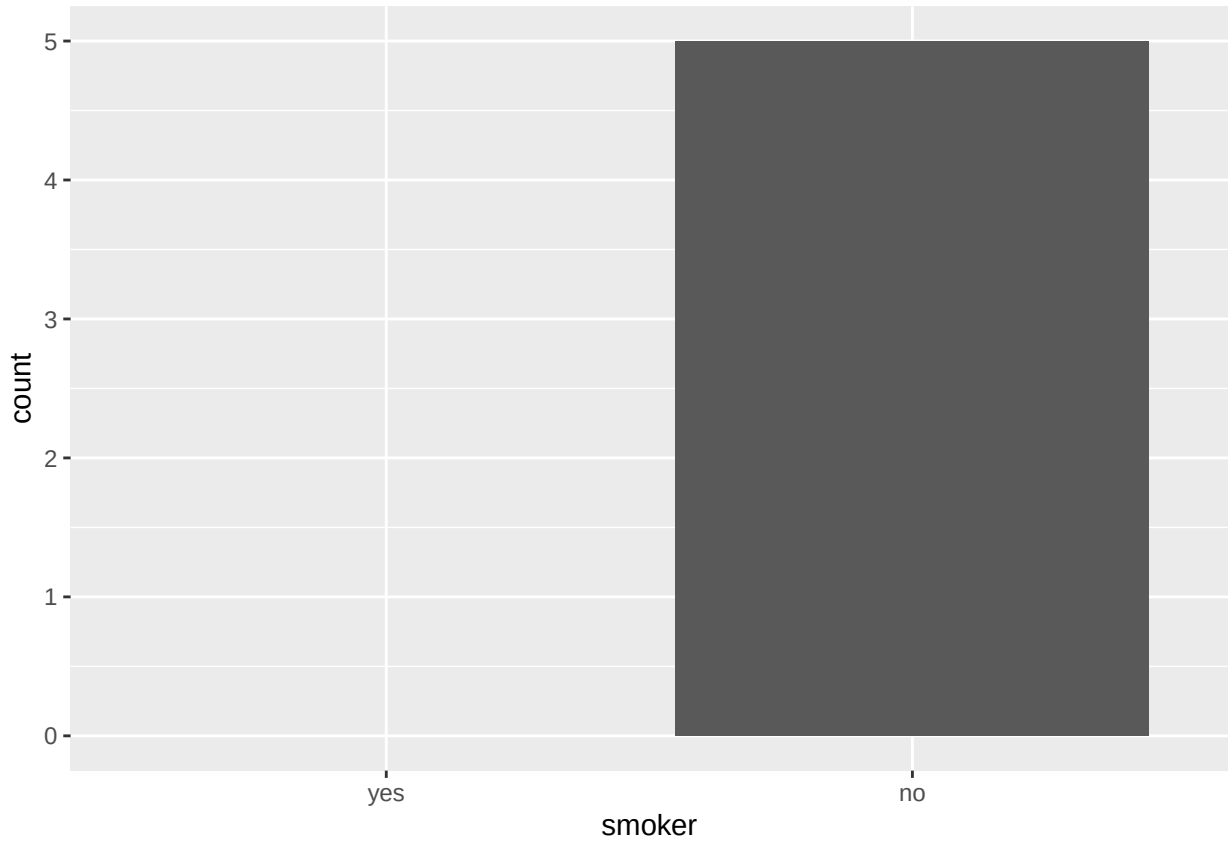
```
## # A tibble: 2 x 2
##   smoker     n
##   <fct>  <int>
## 1 yes        0
## 2 no         5
```

Similar issues can occur when plotting:

```
ggplot(health, aes(x = smoker)) +
  geom_bar() +
  scale_x_discrete()
```




```
ggplot(health, aes(x = smoker)) +
  geom_bar() +
  scale_x_discrete(drop = FALSE)
```




Similar issues can occur when grouping. Again the solution is to supply the `.drop = FALSE` argument:

```
health |>
  group_by(smoker, .drop = FALSE) |>
  summarize(
    n = n(),
    mean_age = mean(age),
    min_age = min(age),
    max_age = max(age),
    sd_age = sd(age)
  )
```

```
## Warning: There were 2 warnings in `summarize()`.
## The first warning was:
## i In argument: `min_age = min(age)`.
## i In group 1: `smoker = yes`.
## Caused by warning in `min()`:
## ! no non-missing arguments to min; returning Inf
## i Run `dplyr::last_dplyr_warnings()` to see the 1 remaining warning.
```

```
## # A tibble: 2 x 6
##   smoker     n mean_age min_age max_age sd_age
##   <fct>  <int>    <dbl>   <dbl>   <dbl>  <dbl>
## 1 yes        0      NaN     Inf    -Inf   NA
## 2 no         5       60      34      88   21.6
```

We got warnings and "interesting" results when we summarized the empty group. Empty vectors have `0` length, while missing values have length of `1`.

To avoid these results (and the warnings!), we can use `complete()` to make the implicit missing values explicit:

```
health |>
  group_by(smoker) |>
  summarize(
    n = n(),
    mean_age = mean(age),
    min_age = min(age),
    max_age = max(age),
    sd_age = sd(age)
  ) |>
  complete(smoker)
```

```
## # A tibble: 2 x 6
##   smoker     n mean_age min_age max_age sd_age
##   <fct>  <int>    <dbl>   <dbl>   <dbl>  <dbl>
## 1 yes       NA       NA      NA      NA   NA
## 2 no         5       60      34      88   21.6
```

The main drawback of this method: the results of `count` was also rendered `NA` instead of 0 for the missing group.